

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

PS F:\Desktop\CODEFORCES> cd "f:\Desktop\CODEFORCES\" ; if ($?) { g
First
Second
PS F:\Desktop\CODEFORCES>
```

In our C++ program, we have been using namespace std, so there is one namespace std that has the cin and cout objects. So that's why we just write a single line using namespace std; and we can use cin and cout objects. Otherwise, we have to write like this,

`std::cout << "Hello";`

Destructors in C++ :-

A destructor in C++ is also a member function like a constructor which is also invoked automatically when the object goes out of scope or we can also explicitly call the destructor to destroy the object by making a call to delete.

A destructor has the same name as the class name as a constructor, but to differentiate between them the destructor function is preceded by a tilde (~) symbol

```
class Test{
    public:
        Test(){
            cout << "Test Object Created" << endl;
        }
        ~Test(){
            cout << "Test Object Destroyed" << endl;
        }
};
```

When the Destructor function will be Called in C++?

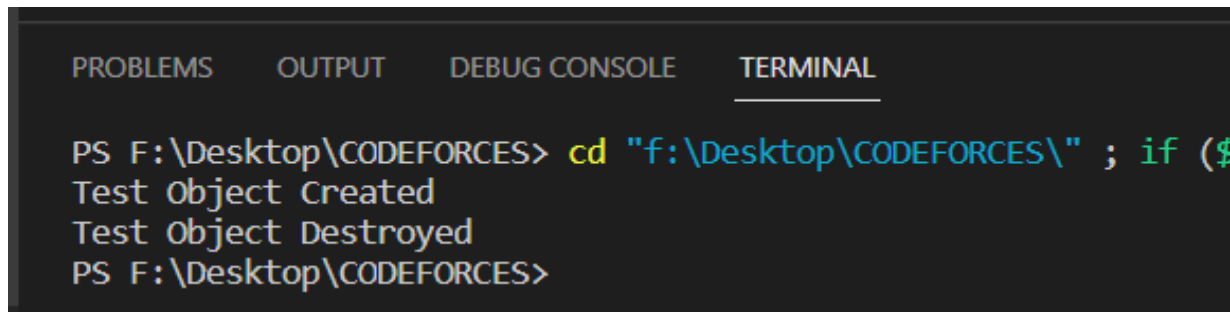
This destructor function will be called when the object will be destroyed. We know once the main function ends, automatically all the objects that are associated with the program get destroyed. **So, the constructor is called when the object is created and the destructor is called when the object is destroyed.**

```
Test *obj = new Test();  
delete obj;
```

As you can see in the above example, we have written `delete obj`. So, when we are creating an object by writing a `new Test()`, the constructor will be called. And when we delete that object by writing `delete obj` then destructor will be called.

```
1  #include <iostream>  
2  using namespace std;  
3  class Test{  
4  public:  
5      Test(){  
6          cout<<"Test Object Created" << endl;  
7      }  
8      ~Test(){  
9          cout<<"Test Object Destroyed" << endl;  
10     }  
11 };  
12 int main(){  
13     Test *obj = new Test();  
14     delete obj;  
15     return 0;  
16 }
```

Output:-



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

PS F:\Desktop\CODEFORCES> cd "f:\Desktop\CODEFORCES\" ; if ($?) { g++ test.cpp -std=c++11 -o test.exe ; test.exe }
Test Object Created
Test Object Destroyed
PS F:\Desktop\CODEFORCES>
```

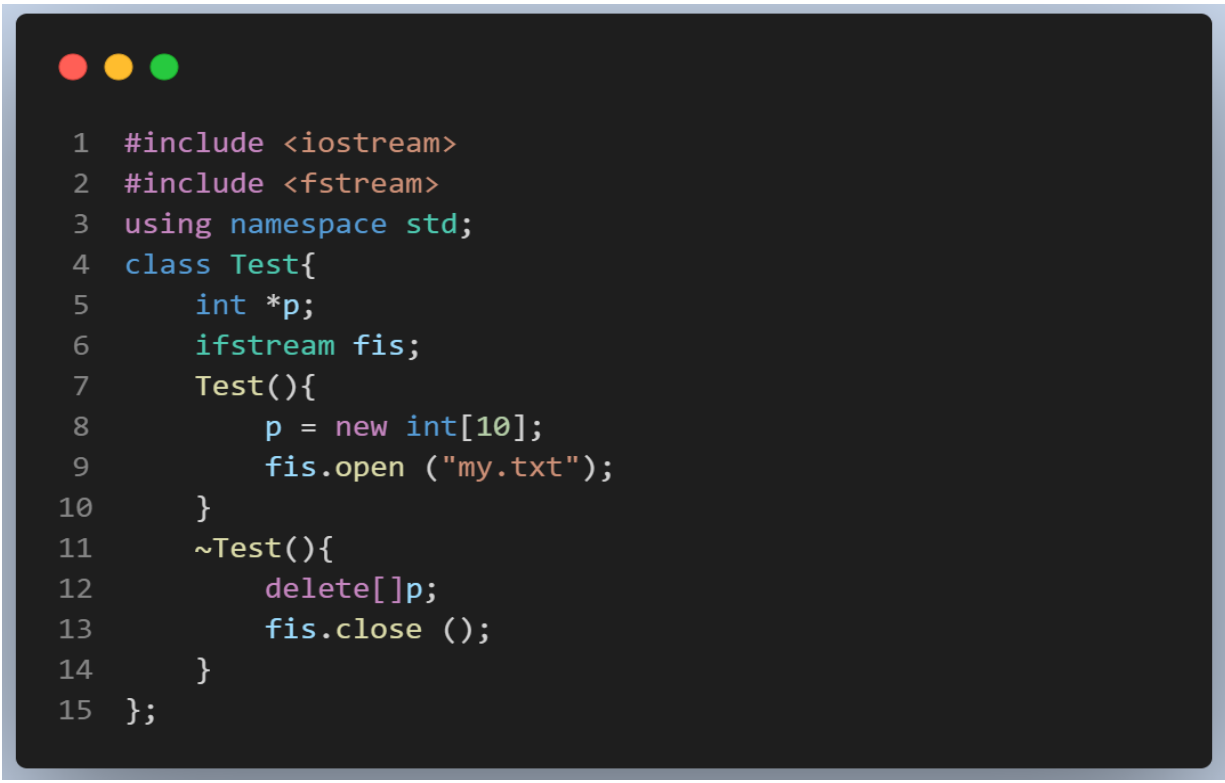
If an object is created dynamically then it should also be deleted when not required. So, when we say `delete obj`, the destructor function will be called.

Is it Mandatory to Define a Destructor in C++?

No, it is not mandatory to define a destructor in C++. If as a programmer we do not define a destructor, the compiler will provide a default one automatically. And for many classes, this compiler-provided default destructor is sufficient. **We only need to define a destructor explicitly in our class when the class handles external resources that need to be released.**

What is the use of Destructor in C++?

The constructor is used for initialization purposes. It is also used for allocating resources. Similarly a destructor is used for deallocating resources or releasing resources. we have to deallocate the resources. Which type of resources? When you are acquiring any external things like heap memory, file, network connection, etc. these are all external sources.



```
1  #include <iostream>
2  #include <fstream>
3  using namespace std;
4  class Test{
5      int *p;
6      ifstream fis;
7      Test(){
8          p = new int[10];
9          fis.open ("my.txt");
10     }
11     ~Test(){
12         delete[]p;
13         fis.close ();
14     }
15 };
```

Here we have a class called Test. Inside this Test class, we have a pointer p. We have allocated this p dynamically inside the constructor. Then inside the destructor, we must delete the allocated memory otherwise this will cause memory leak problems.

Can we overload the constructor? Can we have multiple constructors? Yes. Can we have multiple destructors? No. Can the constructor or destructor return anything? No. All the rules of constructors are followed on destructor except destructor cannot be overloaded. A destructor can be virtual also.

Example to Understand Static Allocation of an object in C++

```
1  #include <iostream>
2  using namespace std;
3  class Test{
4  public:
5      Test(){
6          cout<<"Test Created"<<endl;
7      }
8      ~Test(){
9          cout<<"Test Destroyed"<<endl;
10     }
11 };
12 void fun(){
13     Test t;
14 }
15 int main(){
16     fun();
17 }
```

Output:-

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS F:\Desktop\CODEFORCES> cd "f:\Desktop\CODEFORCES\" ; if (
Test Created
Test Destroyed
PS F:\Desktop\CODEFORCES>
```

Example to Understand Dynamic Allocation of Object in C++

```
1  #include <iostream>
2  using namespace std;
3  class Test{
4  private:
5      int *p;
6  public:
7      Test(){
8          p = new int[10];
9          cout<<"Test Created"<<endl;
10     }
11     ~Test(){
12         delete[]p;
13         cout<<"Test Destroyed"<<endl;
14     }
15 };
16 void fun(){
17     Test *t = new Test ();
18     delete t;
19 }
20 int main(){
21     fun();
22 }
23
```

Output:-

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

PS F:\Desktop\CODEFORCES> cd "f:\Desktop\CODEFORCES\" ; if ($?) {
Test Created
Test Destroyed
PS F:\Desktop\CODEFORCES> 
```

Note :- If we are not writing `delete []p` then the destructor will not be called.

Rules to be Followed While Declaring Destructors in C++:-

- C++ Destructors do not accept arguments.
- You cannot return value from a destructor even if a void is not allowed. So, the destructor should not have any return type.
- Destructors in C++ cannot be declared const, volatile, or static. **However, destructors in C++ can be invoked for the destruction of objects which are declared as const, volatile, or static.**
- Destructors in C++ can be declared as virtual.

When Destructor Function Called in C++?

- A local (automatic) object with block scope goes out of scope.
- An object allocated using the new operator is explicitly deallocated using delete.
- The lifetime of a temporary object ends.
- A program ends and global or static objects exist.
- The destructor is explicitly called using the destructor function's fully qualified name.

Virtual Destructors in C++ :-

Example to Understand Destructors inheritance in C++:

```
1  #include <iostream>
2  using namespace std;
3  class Base{
4  public:
5      Base(){
6          cout<<"Base Class Constructor"<<endl;
7      }
8      ~Base(){
9          cout<<"Base Class Destructor"<<endl;
10     }
11 };
12 class Derived:public Base{
13 public:
14     Derived(){
15         cout<<"Derived Class Constructor"<<endl;
16     }
17     ~Derived(){
18         cout<<"Derived Class Destructor"<<endl;
19     }
20 };
21 int main(){
22     Derived d;
23     return 0;
24 }
```

Output :-


```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

PS F:\Desktop\CODEFORCES> cd "f:\Desktop\CODEFORCES\" ; if ($?) {
Base Class Constructor
Derived Class Constructor
Derived Class Destructor
Base Class Destructor
PS F:\Desktop\CODEFORCES>
```

Here, we have a Base class. It is having two public members functions that are Base() and ~Base() i.e. one public constructor and one public destructor.

Next, we have a Derived class which is inherited from the Base class. It is also having two public member functions that are constructor and destructor.

How are the Destructors Called in C++?

First, the destructor of the Derived class will be called then the destructor of the Base class will be called. The below messages will be printed on the screen.

Derived Class Destructor

Base Class Destructor

So, if you are creating an object of a derived class and when it is being destroyed, first, the destructor of the Derived class will be called then the destructor of the Base class will be called.

So, in contrast to the constructor, destructors are called from the bottom-up approach.

```

1  #include <iostream>
2  using namespace std;
3  class Base{
4  public:
5      Base(){
6          cout<<"Base Class Constructor"<<endl;
7      }
8      ~Base(){
9          cout<<"Base Class Destructor"<<endl;
10     }
11 };
12 class Derived:public Base{
13 public:
14     Derived(){
15         cout<<"Derived Class Constructor"<<endl;
16     }
17     ~Derived(){
18         cout<<"Derived Class Destructor"<<endl;
19     }
20 };
21 int main(){
22     Base *p = new Derived();
23     delete p;
24     return 0;
25 }

```

Output :-

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

PS F:\Desktop\CODEFORCES> cd "f:\Desktop\CODEFORCES\" ; if ($?) {
Base Class Constructor
Derived Class Constructor
Base Class Destructor
PS F:\Desktop\CODEFORCES>

```

Virtual Destructors in C++:

A Virtual Destructor in C++ is **used to release the memory space which is allocated by the derived class object while deleting the object of the derived class using a base class pointer object.**

Here, inside the Main method, we have a Base class pointer and the object is of the Derived class.

Then on the next line, we are deleting p because dynamically created memory should be deleted when not required using the delete operator.

So new Derived object will be destroyed. Which constructor will be called? Remember in C++, the function calls are depending upon the pointer, not upon the object.

So, the pointer p is of the Base class. So only the Base class destructor will be called. The derived class destructor will not be called.

But suppose you want the destructor of the derived class should also be called and then the Base class destructor called. then we have to write virtual keyword before the name of Base class Destructor as,

```
virtual ~Base ()  
{  
    cout << "Base Class Destructor" << endl;  
}
```

If we write virtual keyword there, then even though the pointer is of Base class and object is of Derived class and when the object is deleted then first, **the destructor of Derived class will be called and then the destructor of Base class will be called.**

```
1  #include <iostream>
2  using namespace std;
3  class Base{
4  public:
5      Base(){
6          cout<<"Base Class Constructor"<<endl;
7      }
8      virtual ~Base(){
9          cout<<"Base Class Destructor"<<endl;
10     }
11 };
12 class Derived:public Base{
13 public:
14     Derived(){
15         cout<<"Derived Class Constructor"<<endl;
16     }
17     ~Derived(){
18         cout<<"Derived Class Destructor"<<endl;
19     }
20 };
21 int main(){
22     Base *p = new Derived();
23     delete p;
24     return 0;
25 }
```

Output:-

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

PS F:\Desktop\CODEFORCES> cd "f:\Desktop\CODEFORCES\" ; if ($?) {
Base Class Constructor
Derived Class Constructor
Derived Class Destructor
Base Class Destructor
PS F:\Desktop\CODEFORCES>
```

So, this is how virtual destructors work.

If you don't make the Base class destructor virtual and you are using a pointer of the Base class and object of the Derived class **then only the resources acquired by the Base class code will be released. But the resources acquired by Derived class code will not be released because that will not execute.**

So, this must also execute. If you are using the Base class pointer and Derived class object then you should make the Base class destructor virtual.

Pure Virtual Destructors in C++ :-

The only difference between Virtual Destructor and Pure Virtual Destructor in C++ is that the **pure virtual destructor will make the class Abstract**, hence we cannot create an object of that class but we can create a pointer of that class.

There is no requirement to implement pure virtual destructors in the derived classes as we are going to create instances of the derived class which are going to be pointed by the Base class pointer.

```

1  #include <iostream>
2  using namespace std;
3  class Base{
4  public:
5      Base(){
6          cout<<"Base Class Constructor"<<endl;
7      }
8      virtual ~Base() = 0;
9  };
10 Base::~~Base(){
11     cout<<"Base Class Destructor"<<endl;
12 }
13 class Derived:public Base{
14 public:
15     Derived(){
16         cout<<"Derived Class Constructor"<<endl;
17     }
18     ~Derived(){
19         cout<<"Derived Class Destructor"<<endl;
20     }
21 };
22 int main(){
23     Base *p = new Derived();
24     delete p;
25     return 0;
26 }

```

Output:-

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```

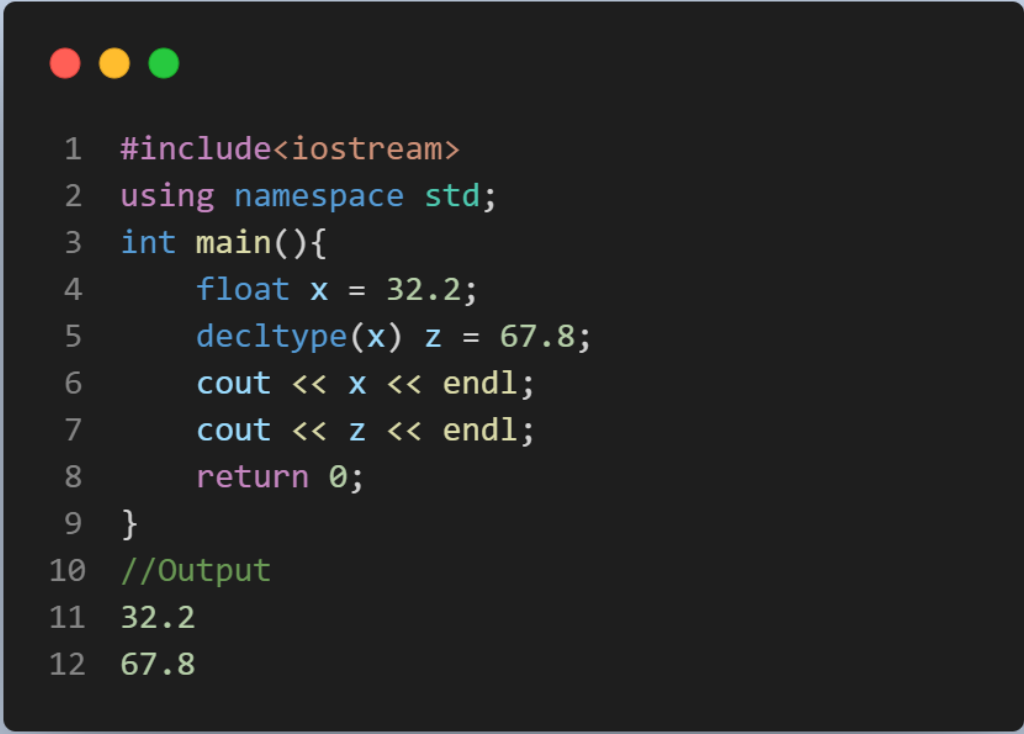
PS F:\Desktop\CODEFORCES> cd "f:\Desktop\CODEFORCES\" ; if ($?) { g++ der
Base Class Constructor
Derived Class Constructor
Derived Class Destructor
Base Class Destructor
PS F:\Desktop\CODEFORCES>

```

Some features of C++ 11

Auto Keyword in C++ :- The auto keyword is a very powerful and useful feature of C++ 11. When a programmer uses the library functions or some functions of the built-in classes then he/she doesn't have to know about the data type. We can simply use the auto declaration to do it automatically. It saves the programmer's time.

Decltype in C++ 11:-



```
1  #include<iostream>
2  using namespace std;
3  int main(){
4      float x = 32.2;
5      decltype(x) z = 67.8;
6      cout << x << endl;
7      cout << z << endl;
8      return 0;
9  }
10 //Output
11 32.2
12 67.8
```

Final Keyword in C++

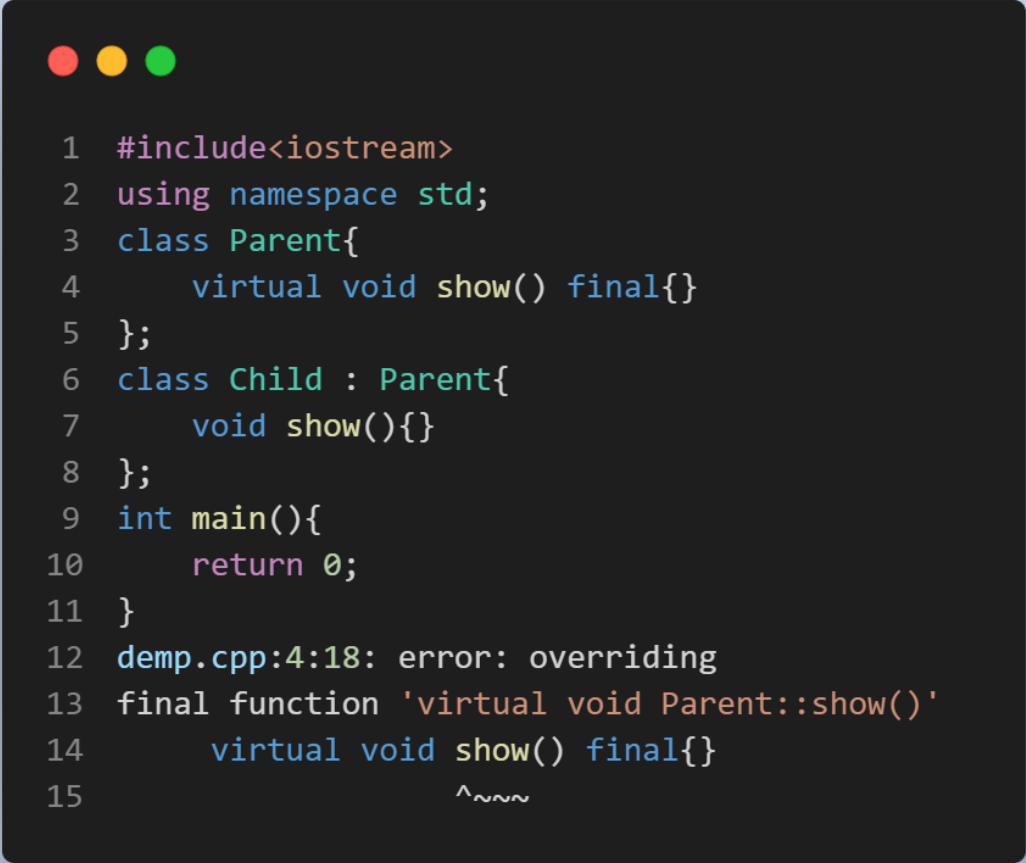
Mainly the final keyword is used for two purposes. They are as follows :-

- Restrict Class Inheritance
- Restrict Method Overriding

Restrict Class Inheritance :-

```
1  #include<iostream>
2  using namespace std;
3  class Parent final{
4      void show(){}
5  };
6  class Child:Parent{
7      void show (){}
8  };
9  int main(){
10     return 0;
11 }
12 Error :-
13 demp.cpp:6:7: error: cannot derive from 'final'
14 base 'Parent' in derived type 'Child'
15 class Child:Parent{
16     ^~~~~~
```


Restrict Method Overriding :-



```
1  #include<iostream>
2  using namespace std;
3  class Parent{
4      virtual void show() final{}
5  };
6  class Child : Parent{
7      void show(){}
8  };
9  int main(){
10     return 0;
11 }
12 demp.cpp:4:18: error: overriding
13 final function 'virtual void Parent::show()'
14     virtual void show() final{}
15                     ^~~~~
```